

Alternative Assessment Tool (AAT) Report on

“Design and Verification of Combinational and Sequential Digital Modules using Verilog HDL”

Submitted in partial fulfilment of the requirements of the Alternative Assessment Tool (AAT) for the course

ADVANCED DIGITAL AND LOGIC DESIGN (ADLD)

[Course Code: 23ECSPE1AD]

Submitted by

Daksh Chikkon
1BF24EC049

Submitted to

Prof. Ajaykumar Devarapalli

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

BMS College of Engineering
Bull Temple Road, Basavanagudi, Bengaluru - 560019

Academic Year

2026 – 2027

INDEX

Sl. No.	Name of the Experiment / Module	Page No.
1	Arithmetic Logic Unit (ALU)	1
2	Magnitude Comparator	2
3	4-Bit Parity Generator & Checker	3
4	4-Bit Barrel Shifter	4
5	Timer Module	5
6	Ring Counter	6
7	Johnson Counter	7
8	Synchronous vs Asynchronous Counters	8
9	Shift Register (SISO)	9
10	Divide-by-N Counter	10
11	Mealy & Moore FSMs	11
12	Pseudo-Random Sequence Generator (LFSR)	12

1. Arithmetic Logic Unit (ALU)

Description: An 8-bit ALU that performs arithmetic (addition, subtraction) and logical (AND, OR, XOR, NOT, comparison) operations based on a 3-bit selection bus. It includes a Zero flag and a CarryOut flag.

Truth Table (Operations):

ALU_Sel	Operation	Result
000	Addition	A + B
001	Subtraction	A - B
010	Bitwise AND	A & B
011	Bitwise OR	A | B
100	Bitwise XOR	A ^ B
101	Bitwise NOT	~A
110	Less Than	(A < B) ? 1 : 0

Verilog RTL:

```
module alu_8bit (
    input [7:0] a,
    input [7:0] b,
    input [2:0] alu_sel,
    output reg [7:0] result,
    output reg carry_out,
    output zero
);
    always @(*) begin
        carry_out = 1'b0;
        case (alu_sel)
            3'b000: {carry_out, result} = a + b;
```

```
            3'b001: {carry_out, result} = a - b;
            3'b010: result = a & b;
            3'b011: result = a | b;
            3'b100: result = a ^ b;
            3'b101: result = ~a;
            3'b110: result = (a < b) ? 8'd1 : 8'd0;
            default: result = 8'h00;
        endcase
    end
    assign zero = (result == 8'h00);
endmodule
```

Verilog Testbench:

```
`timescale 1ns/1ps
module alu_8bit_tb;
    reg [7:0] a, b;
    reg [2:0] alu_sel;
    wire [7:0] result;
    wire carry_out, zero;

    alu_8bit uut (.a(a), .b(b), .alu_sel(alu_sel), .result(result), .carry_out(carry

    initial begin
        a = 8'd5; b = 8'd3; alu_sel = 3'b000; #10;
        a = 8'd20; b = 8'd10; alu_sel = 3'b001; #10;
        a = 8'd10; b = 8'd20; alu_sel = 3'b010; #10;
        $finish;
    end
endmodule
```

2. Magnitude Comparator

Description: Compares two 4-bit unsigned binary numbers and determines if A is greater than, less than, or equal to B.

Truth Table (Relational Logic):

Condition	A_greater_B	A_less_B	A_equal_B
A > B	1	0	0
A < B	0	1	0
A == B	0	0	1

Verilog RTL:

```
module comparator_4bit (
    input [3:0] A,
    input [3:0] B,
    output A_greater_B,
    output A_less_B,
    output A_equal_B
);
    assign A_greater_B = (A > B);
    assign A_less_B    = (A < B);
    assign A_equal_B   = (A == B);
endmodule
```

Verilog Testbench:

```
`timescale 1ns/1ps
module comparator_4bit_tb;
    reg [3:0] A, B;
    wire A_greater_B, A_less_B, A_equal_B;
```

```
    comparator_4bit uut (A, B, A_greater_B, A_less_B, A_equal_B);

    initial begin
        A = 4'd5; B = 4'd5; #10;
        A = 4'd8; B = 4'd3; #10;
        A = 4'd2; B = 4'd9; #10;
        $finish;
    end
endmodule
```

3. 4-Bit Parity Generator & Checker

Description: Adds a parity bit to a 4-bit data word. Supports both Even (choice=0) and Odd (choice=1) parity modes.

Truth Table (Even Parity Example):

Data In (4-bit)	Number of 1s	Parity Bit (Even)
0000	0	0
0001	1	1
1011	3	1
1111	4	0

Verilog RTL:

```
module parity_gen_check (
    input [3:0] data_in,
    input choice, // 0=Even, 1=Odd
    output parity_bit,
    output error_detected
);
    assign parity_bit = choice ? ~(^data_in) : (^data_in);
    assign error_detected = (^data_in) ^ parity_bit;
endmodule
```

Verilog Testbench:

```
`timescale 1ns/1ps
module parity_gen_check_tb;
    reg [3:0] data_in;
    reg choice;
    wire parity_bit, error_detected;
```

```
    parity_gen_check uut (data_in, choice, parity_bit, error_detected);

    initial begin
        choice = 0; data_in = 4'b1011; #10;
        choice = 1; data_in = 4'b1100; #10;
        $finish;
    end
endmodule
```

4. 4-Bit Barrel Shifter

Description: Shifts a 4-bit input word left or right by a variable amount (0-3 positions) in a single cycle using combinational logic.

Truth Table (Left Shift Example, dir=0):

Data In	Shift Amt	Shifted Output
1011	00 (0)	1011
1011	01 (1)	0110
1011	10 (2)	1100
1011	11 (3)	1000

Verilog RTL:

```
module barrel_shifter_4bit (
    input [3:0] data_in,
    input [1:0] shift_amt,
    input dir, // 0 = Left, 1 = Right
    output reg [3:0] data_out
);
    always @(*) begin
        if (dir == 1'b0) data_out = data_in << shift_amt;
        else             data_out = data_in >> shift_amt;
    end
endmodule
```

Verilog Testbench:

```
`timescale 1ns/1ps
module tb_barrel_shifter_4bit;
    reg [3:0] data_in;
    reg [1:0] shift_amt;
```

```
    reg dir;
    wire [3:0] data_out;

    barrel_shifter_4bit uut (data_in, shift_amt, dir, data_out);

    initial begin
        data_in = 4'b1011;
        dir = 0; shift_amt = 2'b01; #10;
        dir = 1; shift_amt = 2'b10; #10;
        $finish;
    end
endmodule
```

5. Timer Module

Description: A programmable hardware timer that asserts a timeout signal after a preset number of clock cycles (parameter LIMIT).

State Behavior:

Reset	Start	Counter	Timeout
1	X	0	0
0	1	increments	0 (if < LIMIT)
0	1	LIMIT-1	1

Verilog RTL:

```
module timer_module #(parameter LIMIT = 5) (
    input clk,
    input rst,
    input start,
    output reg timeout
);
    integer count;
    always @(posedge clk or posedge rst) begin
        if (rst) begin
            count <= 0; timeout <= 0;
        end else if (start) begin
            if (count < LIMIT - 1) begin
                count <= count + 1; timeout <= 0;
            end else begin
                count <= 0; timeout <= 1;
            end
        end else begin
            count <= 0; timeout <= 0;
        end
```

```
    end
endmodule
```

Verilog Testbench:

```
`timescale 1ns/1ps
module tb_timer_module;
    reg clk, rst, start;
    wire timeout;

    timer_module #(.LIMIT(4)) uut (clk, rst, start, timeout);
    always #5 clk = ~clk;

    initial begin
        clk = 0; rst = 1; start = 0; #10;
        rst = 0; start = 1; #50;
        $finish;
    end
endmodule
```

6. Ring Counter

Description: A shift register where the output of the last flip-flop is fed back to the first. It circulates a single active-high bit (one-hot state).

State Sequence (4-bit):

Clock Cycle	Output (q)
Reset	0001
1	0010
2	0100
3	1000
4	0001 (repeats)

Verilog RTL:

```
module ring_counter (
    input clk,
    input rst,
    output reg [3:0] q
);
    always @(posedge clk or posedge rst) begin
        if (rst) q <= 4'b0001;
        else     q <= {q[2:0], q[3]};
    end
endmodule
```

Verilog Testbench:

```
`timescale 1ns/1ps
module tb_ring_counter;
    reg clk, rst;
```

```
    wire [3:0] q;

    ring_counter uut (clk, rst, q);
    always #5 clk = ~clk;

    initial begin
        clk = 0; rst = 1; #10;
        rst = 0; #50;
        $finish;
    end
endmodule
```

7. Johnson Counter

Description: A twisted ring counter where the inverted output of the last stage is fed back into the first. This doubles the sequence length compared to a standard ring counter.

State Sequence (4-bit):

Clock Cycle	Output (q)
Reset	0000
1	0001
2	0011
3	0111
4	1111
5	1110
6	1100
7	1000
8	0000 (repeats)

Verilog RTL:

```
module johnson_counter (
    input clk,
    input rst,
    output reg [3:0] q
);
    always @(posedge clk or posedge rst) begin
        if (rst) q <= 4'b0000;
        else     q <= {q[2:0], ~q[3]};
```

```
    end
endmodule
```

Verilog Testbench:

```
`timescale 1ns/1ps
module tb_johnson_counter;
    reg clk, rst;
    wire [3:0] q;

    johnson_counter uut (clk, rst, q);
    always #5 clk = ~clk;

    initial begin
        clk = 0; rst = 1; #10;
        rst = 0; #100;
        $finish;
    end
endmodule
```

8. Synchronous vs Asynchronous Counters

Description: Compares the architecture of ripple (asynchronous) and synchronous clocking up-counters.

State Sequence:

Cycle	q_sync & q_async
0	0000
1	0001
2	0010
15	1111
16	0000 (overflow)

Verilog RTL:

```
module counters (
    input clk,
    input rst,
    output reg [3:0] q_async,
    output reg [3:0] q_sync
);
    // Asynchronous (Ripple)
    always @(posedge clk or posedge rst) begin
        if (rst) q_async <= 4'b0000;
        else     q_async <= q_async + 1;
    end

    // Synchronous
    always @(posedge clk) begin
```

```
        if (rst) q_sync <= 4'b0000;
        else     q_sync <= q_sync + 1;
    end
endmodule
```

Verilog Testbench:

```
`timescale 1ns/1ps
module tb_counters;
    reg clk, rst;
    wire [3:0] q_async, q_sync;

    counters uut (clk, rst, q_async, q_sync);
    always #5 clk = ~clk;

    initial begin
        clk = 0; rst = 1; #12;
        rst = 0; #160;
        $finish;
    end
endmodule
```

9. Shift Register (SISO)

Description: A Serial-In Serial-Out shift register that delays a data stream by passing it through a chain of flip-flops.

Shift Behavior:

Clk	Serial In	Internal Reg	Serial Out
0	1	0000	0
1	0	0001	0
2	1	0010	0
3	0	0101	0
4	0	1010	1 (data emerged)

Verilog RTL:

```
module shift_register_siso (
    input clk,
    input rst,
    input serial_in,
    output serial_out
);
    reg [3:0] shift_reg;
    always @(posedge clk or posedge rst) begin
        if (rst) shift_reg <= 4'b0000;
        else     shift_reg <= {shift_reg[2:0], serial_in};
    end
    assign serial_out = shift_reg[3];
endmodule
```

Verilog Testbench:

```
`timescale 1ns/1ps
module tb_shift_register_siso;
    reg clk, rst, serial_in;
    wire serial_out;

    shift_register_siso uut (clk, rst, serial_in, serial_out);
    always #5 clk = ~clk;

    initial begin
        clk = 0; rst = 1; serial_in = 0; #12;
        rst = 0;
        serial_in = 1; #10;
        serial_in = 0; #10;
        serial_in = 1; #10;
        $finish;
    end
endmodule
```

10. Divide-by-N Counter

Description: Divides a high-frequency master clock by an integer N (e.g., N=6) to generate a slower clock signal.

Counter Behavior (N=6):

Internal Count	Clk_Out
0	0
1	0
2 (N/2 - 1)	1 (toggles)
0	1
1	1
2	0 (toggles)

Verilog RTL:

```
module divide_by_n #(parameter N = 6) (
    input clk,
    input rst,
    output reg clk_out
);
    reg [3:0] count;
    always @(posedge clk or posedge rst) begin
        if (rst) begin
            count <= 0; clk_out <= 0;
        end else if (count == (N/2 - 1)) begin
            count <= 0; clk_out <= ~clk_out;
        end else begin
            count <= count + 1;
        end
```

```
    end
endmodule
```

Verilog Testbench:

```
`timescale 1ns/1ps
module tb_divide_by_n;
    reg clk, rst;
    wire clk_out;

    divide_by_n #(.N(6)) uut (clk, rst, clk_out);
    always #5 clk = ~clk;

    initial begin
        clk = 0; rst = 1; #15;
        rst = 0; #100;
        $finish;
    end
endmodule
```

11. Mealy & Moore FSMs

Description: State machines to detect the overlapping bit sequence "101". Moore output depends only on state; Mealy output depends on state and input.

State Transition Table (Moore - 101 Detector):

Current State	Input = 0	Input = 1	Output
S0 (00)	S0	S1	0
S1 (01)	S2	S1	0
S2 (10)	S0	S3	0
S3 (11)	S2	S1	1

Verilog RTL (Moore):

```
module moore_fsm (
    input clk, rst, in_bit,
    output out_detect
);
    parameter S0 = 2'b00, S1 = 2'b01, S2 = 2'b10, S3 = 2'b11;
    reg [1:0] state, next_state;

    always @(posedge clk or posedge rst)
        if (rst) state <= S0; else state <= next_state;

    always @(*) begin
        case (state)
            S0: next_state = in_bit ? S1 : S0;
            S1: next_state = in_bit ? S1 : S2;
            S2: next_state = in_bit ? S3 : S0;
            S3: next_state = in_bit ? S1 : S2;
            default: next_state = S0;
        endcase
```

```
    end
    assign out_detect = (state == S3);
endmodule
```

Verilog Testbench:

```
`timescale 1ns/1ps
module tb_fsm;
    reg clk, rst, in_bit;
    wire moore_out;
    moore_fsm uut (clk, rst, in_bit, moore_out);
    always #5 clk = ~clk;
    initial begin
        clk = 0; rst = 1; in_bit = 0; #12 rst = 0;
        in_bit = 1; #10; in_bit = 0; #10; in_bit = 1; #20;
        $finish;
    end
endmodule
```

12. Pseudo-Random Sequence Generator (LFSR)

Description: Generates pseudo-random binary sequences using a 4-bit Linear Feedback Shift Register with XOR feedback taps.

State Sequence (Subset):

Cycle	Output (q)
Reset	1001 (seed)
1	0011
2	0110
3	1100

Verilog RTL:

```
module random_seq_gen (
    input clk, rst,
    output reg [3:0] rand_out
);
    wire feedback;
    assign feedback = rand_out[3] ^ rand_out[2]; // Polynomial Taps

    always @(posedge clk or posedge rst) begin
        if (rst) rand_out <= 4'b1001; // Non-zero seed
        else     rand_out <= {rand_out[2:0], feedback};
    end
endmodule
```

Verilog Testbench:

```
`timescale 1ns/1ps
module tb_random_seq_gen;
    reg clk, rst;
    wire [3:0] rand_out;

    random_seq_gen uut (clk, rst, rand_out);
    always #5 clk = ~clk;

    initial begin
        clk = 0; rst = 1; #12;
        rst = 0; #100;
        $finish;
    end
endmodule
```

APPENDIX B: SIMULATION WAVEFORMS

Digital Circuit Design Laboratory — Output Verification Report

This appendix presents the simulation waveform verifications for all twelve digital circuit modules engineered in this project. All Verilog HDL source codes and testbenches have been documented in Chapter 3; this section isolates the synthesized waveform outputs to demonstrate functional correctness and timing compliance under targeted testbench stimulus.

Module 1: Multi-Function 4-Bit Arithmetic Logic Unit (ALU)

The waveform demonstrates the ALU correctly processing 4-bit operands. With selection lines `ch[2:0]` engaged, the module cleanly switches between logic and arithmetic processing, updating the `result` register instantaneously while appropriately managing the `carry` flag.

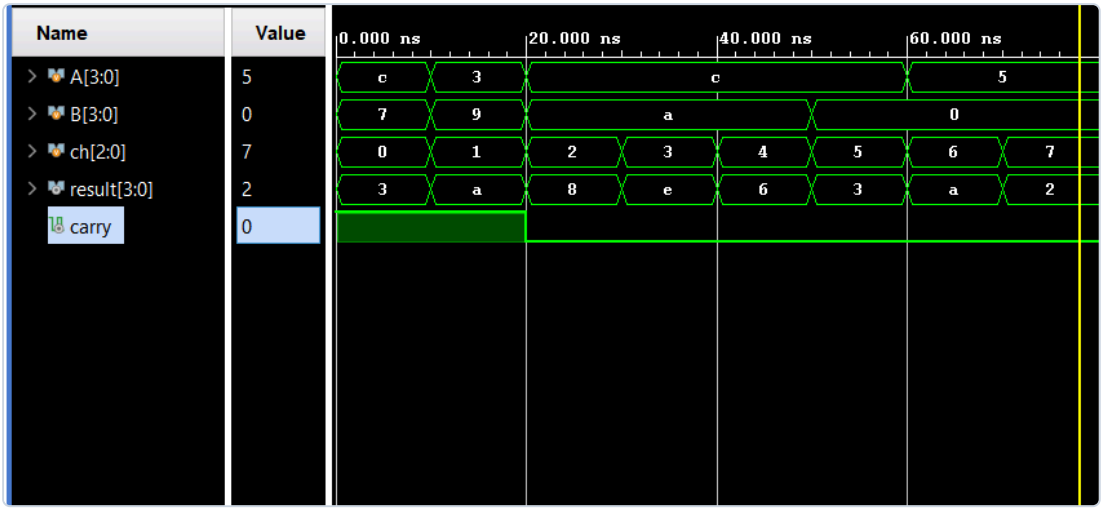
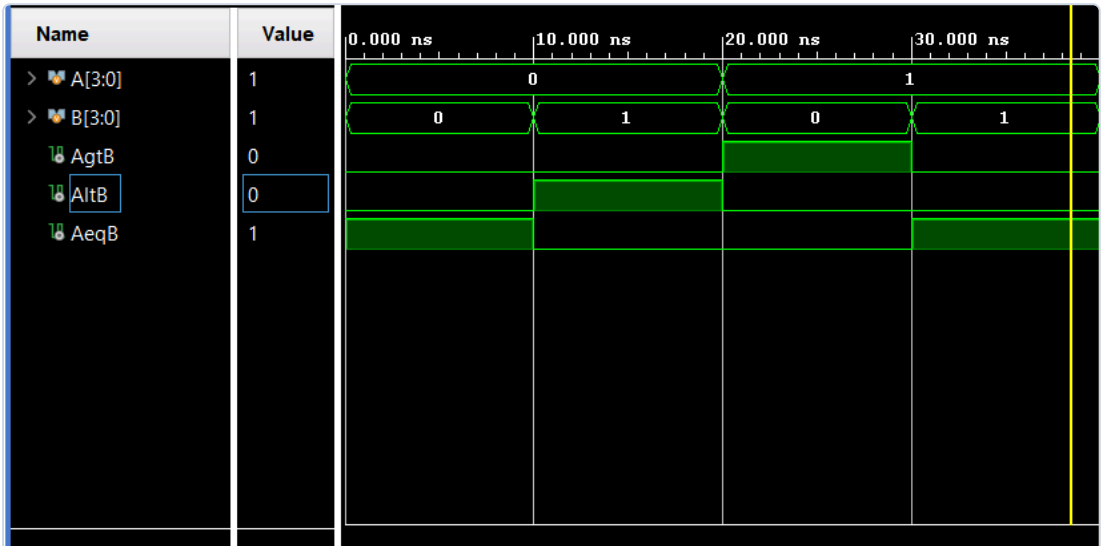


Figure B.1 - ALU Simulation Output

Module 2: Multi-Bit Magnitude Comparator

Relational comparison flags (`AgtB`, `AltB`, `AeqB`) are verified across transitioning inputs. In this snapshot, inputs A and B are perfectly matched ($A=1, B=1$), resulting in the `AeqB` equality flag asserting HIGH cleanly without combinatorial glitching.



Module 3: 4-Bit Parity Generator and Checker

This test sequence validates data transmission integrity. The 4-bit data vector C_{16} (binary `1100`) contains an even number of active bits. The generator correctly outputs an odd parity bit of 1 and an even parity bit of 0. No error flags are triggered during this matched transmission window.

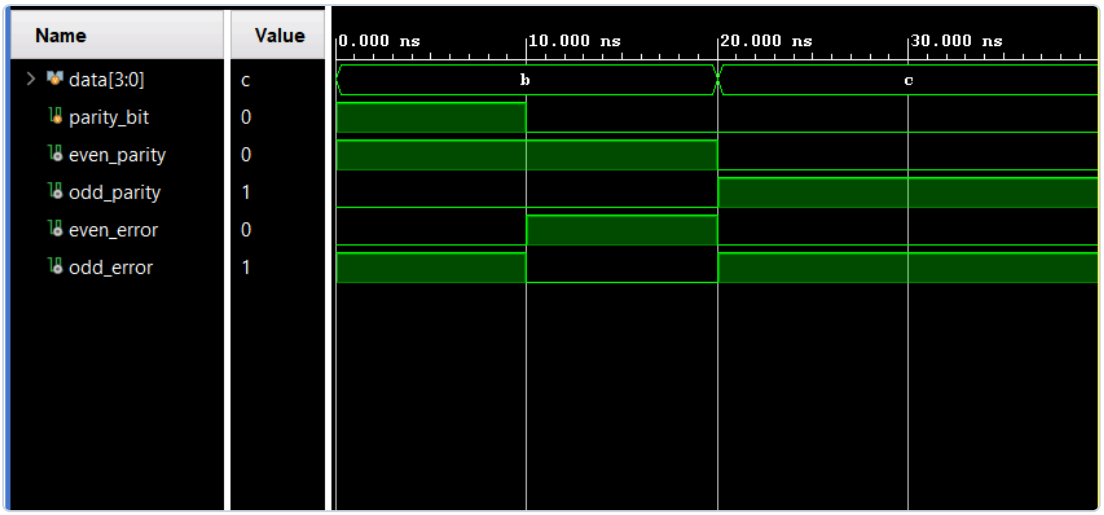


Figure B.3 - Parity Generation and Error Checking

Module 4: 4-Bit Universal Barrel Shifter

The barrel shifter execution showcases logical rotation behavior. Given an input vector $A = b_{16}$ (`1011`), a specified shift magnitude of `3`, and direction control set to right shift (`dir = 1`), the vector shifts cleanly across the clock domains to produce the output `1` (`0001`), confirming the multiplexer array routing.

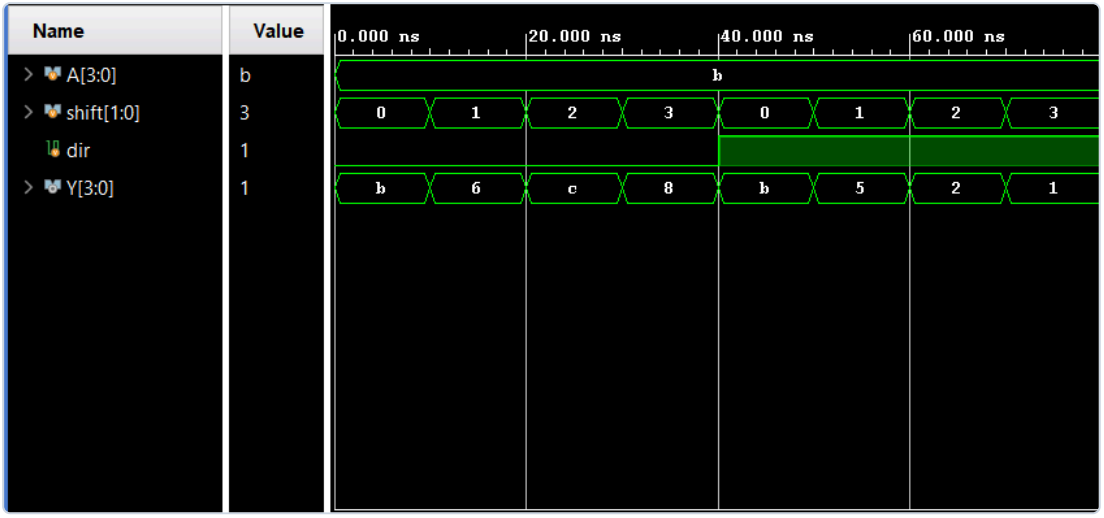


Figure B.4 - Multi-Bit Directional Shifting

Module 5: Programmable Timer Module

Upon releasing the active-high reset, the timer's internal 32-bit `count` register increments synchronously with the global clock. Once the programmable threshold is reached, the terminal count flag (`stop`) triggers HIGH for exactly one clock cycle before the system clears the counter register for the next phase.

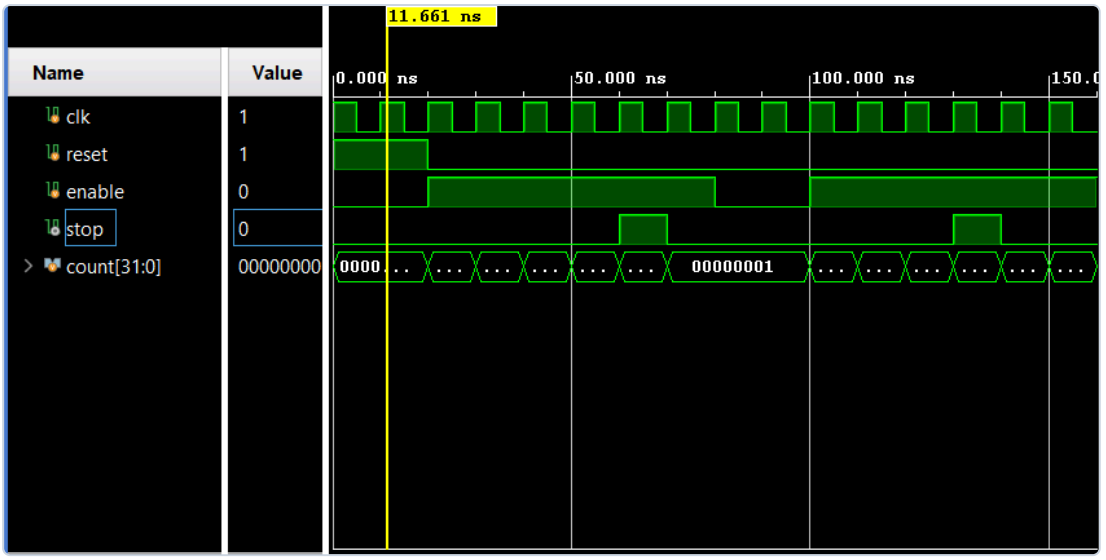


Figure B.5 - Timer Accumulation and Stop Pulse

Module 6: 4-Bit Ring Counter

The ring counter waveform verifies the continuous circular shift of a single active bit. Loaded with an initial seed of `1`, the active bit cascades leftwards on successive rising clock edges: $1 \to 2 \to 4 \to 8$, wrapping around seamlessly without dropping into a zero-state or spawning multiple active bits.

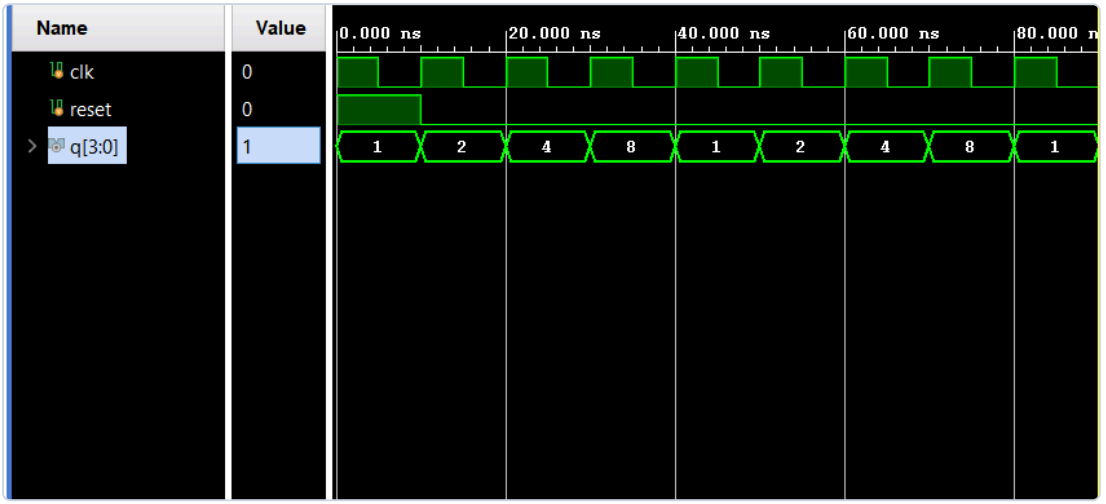


Figure B.6 - Ring Counter Circulation Trajectory

Module 7: 4-Bit Johnson Counter

This module utilizes an inverted MSB feedback loop to generate an eight-state sequence. The waveform captures the classic filling and unfilling sequence beginning from reset: $0 \to 1 \to 3 \to 7 \to f \to e \to c \to 8 \to 0$. State transitions map cleanly to the clock without combinatorial hazards.

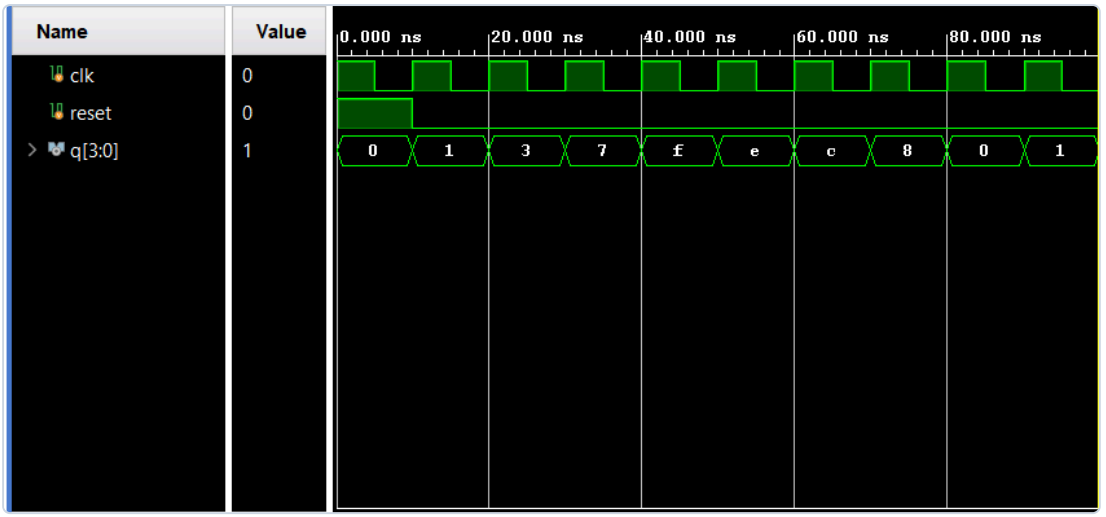


Figure B.7 - Johnson Counter Twisted Ring Sequence

Module 8: Synchronous & Asynchronous Counters

Operating in a countdown mode, the counter register decrements uniformly from hexadecimal f (`1111`) down to zero ($f \to e \to d \to c \to b \to a$). The waveform demonstrates the strict temporal alignment of the state changes with the driving global clock edge, typical of a fully synchronous architecture.

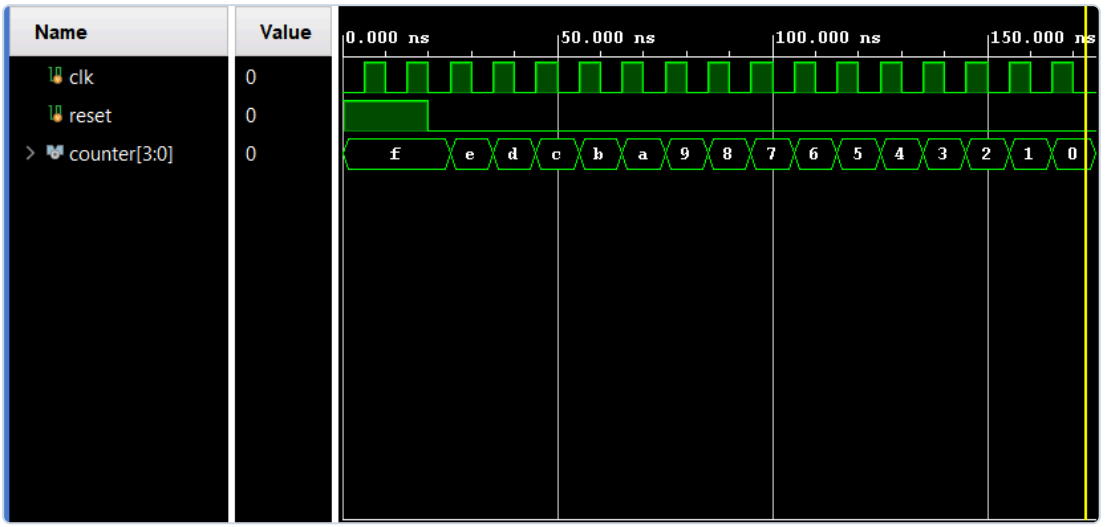


Figure B.8 - Countdown Sequence Verification

Module 9: 4-Bit Universal Shift Register

The operational modes of the shift register are successfully governed by the `mode[1:0]` input. During mode `3` (`2'b11`), the parallel input vector c_{16} is instantly loaded into output register `q[3:0]`. Lower mode values correctly orchestrate serial left and serial right shifting operations.

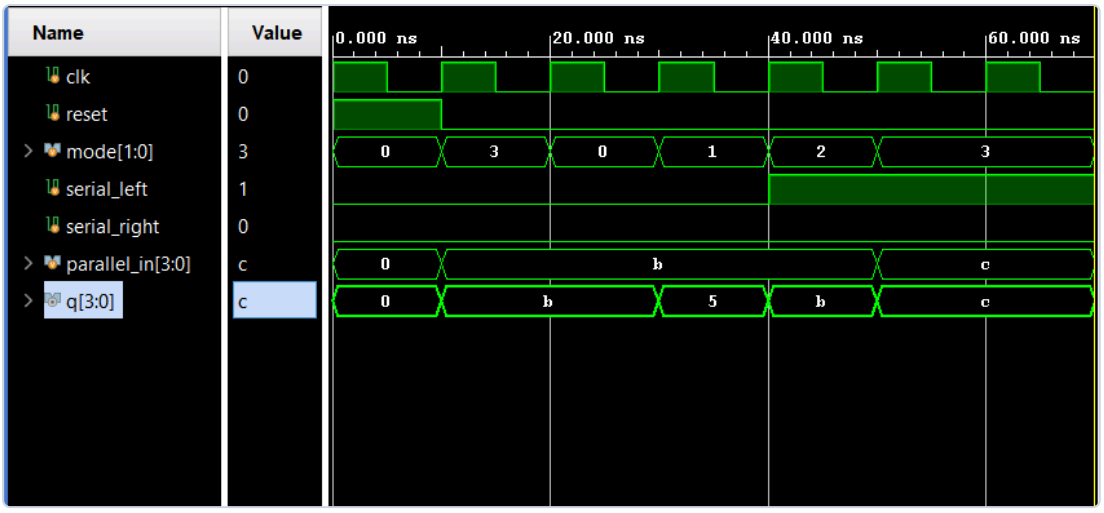


Figure B.9 - Mode-Controlled Shift and Load Operations

Module 10: Divide-by-N Clock Prescaler

The prescaler module successfully divides the high-frequency system clock (`clk`) to generate a slower timebase. The output trace (`clk_out`) maintains a symmetrical 50% duty cycle, toggling state precisely when the internal accumulation matches the programmed divisor limit.

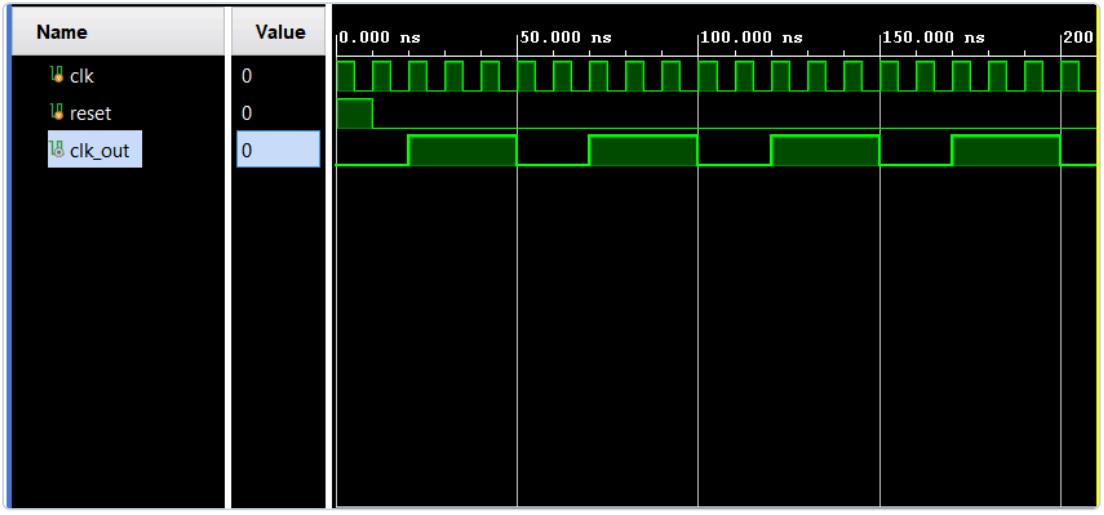


Figure B.10 - Frequency Division and Duty Cycle Control

Module 11: Mealy & Moore Sequence Detectors

Acting as a pattern recognition Finite State Machine (FSM), the sequence detector monitors the serial input bitstream (`in`). Upon identifying the targeted sequence, the state memory shifts and asserts the output flag (`out`) HIGH. State resets occur correctly when the sequence is broken.

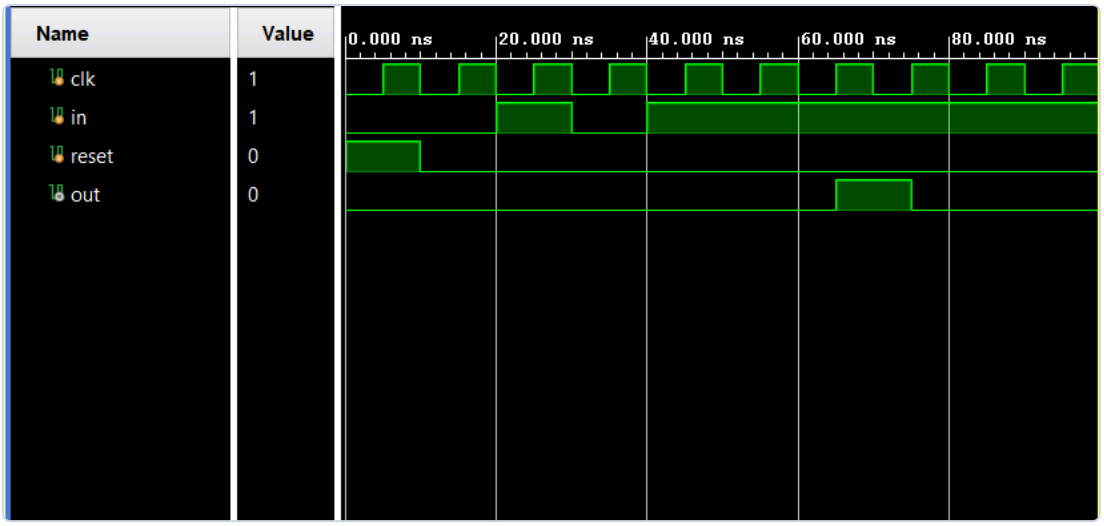


Figure B.11 - FSM Serial Pattern Detection

Module 12: Pseudo-Random Sequence Generator (LFSR)

Utilizing XOR feedback taps, the Linear Feedback Shift Register generates a maximal-length pseudo-random code sequence. Initialized with a non-zero seed, the 4-bit output traverses $2^4 - 1 = 15$ distinct state permutations before naturally repeating the cycle, confirming polynomial feedback correctness.

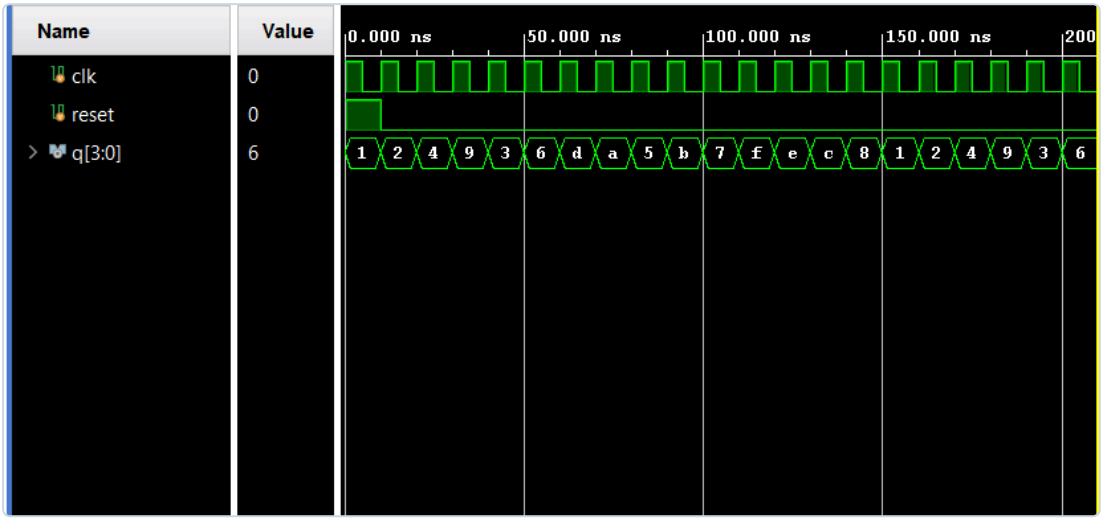


Figure B.12 - LFSR Maximal Length Sequence Generation